

FaltaUno — Protocolo Oficial de Git & GitHub

Flujo de Trabajo para el Equipo de Desarrollo (6 Integrantes) • RIWI 2026

¡Equipo! Para asegurar que el desarrollo de nuestro MVP sea fluido, rápido y libre de códigos sobreescritos, adoptaremos una estrategia basada en **Git Flow Simplificado** utilizando un repositorio centralizado único (Monorepo).

1. Arquitectura de Ramas Operativa

Trabajaremos con tres niveles de ramas estrictos. Nadie tiene permitido subir cambios directamente a la producción sin revisión.

- **main** **Rama de Producción:** Contiene el código 100% estable y verificado. Es la rama oficial que evaluarán los mentores de RIWI al final del proyecto.
- **develop** **Rama de Integración:** Aquí se une el trabajo diario de todos. Debe compilar perfectamente en todo momento.
- **feature/** **Ramas de Tareas:** Ramas temporales creadas para desarrollar una funcionalidad específica vinculada a Jira.

2. El Ciclo Diario del Desarrollador (Paso a Paso)

Cada vez que inicies una nueva tarea asignada en el tablero de Jira, debes seguir rigurosamente este protocolo en tu terminal:

Paso A: Actualizar tu entorno local

Antes de escribir una sola línea de código, asegúrate de tener lo último que tus compañeros han integrado:

```
git checkout develop
git pull origin develop
```

Paso B: Crear tu rama de característica

Crea una rama descriptiva basada en la tarea. Utiliza la nomenclatura acordada (ej. **feature/login-auth** o **feature/muro-partidos**):

```
git checkout -b feature/nombre-de-tu-tarea
```

Paso C: Confirmar tus avances locales

Realiza commits frecuentes con mensajes claros en tiempo presente:

```
git add .
git commit -m "Agrega validaciones del cliente en el formulario de registro"
```

3. Regla de Oro: Evitar Conflictos antes de Subir

Para evitar romper el código general en GitHub, **es obligatorio** fusionar lo último de `develop` en tu propia rama local antes de enviar tu Pull Request:

```
# Estando en tu rama feature  
git pull origin develop
```

Si Git detecta un conflicto, resuélvelo directamente en tu editor de código, asegúrate de que el proyecto levante sin errores, realiza un nuevo commit y procede al siguiente paso.

⚠ ALERTA DE CONFLICTOS

Si un archivo es editado por dos personas simultáneamente, Git detendrá el proceso. Es responsabilidad del desarrollador resolver el conflicto en local antes de subir cambios a la nube. En caso de dudas, consulta inmediatamente con el Scrum Master.

4. Publicación y Pull Request (PR)

Cuando la tarea esté completamente desarrollada y testeada localmente, súbela a GitHub:

```
git push origin feature/nombre-de-tu-tarea
```

Posteriormente, ingresa a la interfaz web de GitHub y abre un **Pull Request (PR)** solicitando la fusión desde tu rama hacia `develop`.

💡 PROCESO DE APROBACIÓN

El Scrum Master y el Analista/QA revisarán el código en GitHub. Una vez verificado el cumplimiento de los criterios de aceptación y las validaciones correspondientes, el Scrum Master realizará el **Merge** definitivo en la nube.